
Introduction to the Theory of Computation

计算理论基础

胡晓鹏

xphu@swjtu.edu.cn

Course Auxiliary Utilities

课程QQ群



群名称: 2026计算理论基础
群 号: 386077542



课堂名称: 计算理论基础

课堂编号: SC543

- 1、扫码关注公众号: [微助教服务号](#)。
- 2、点击系统通知: “[点击此处加入【计算理论基础】课堂](#)”, 填写学生资料加入课堂。

*如未成功收到系统通知, 请点击公众号下方 “学生” - “全部(A)” - “加入课堂” --- “输入课堂编号” 手动加入课堂

二维码有效期至: 2026-10-14

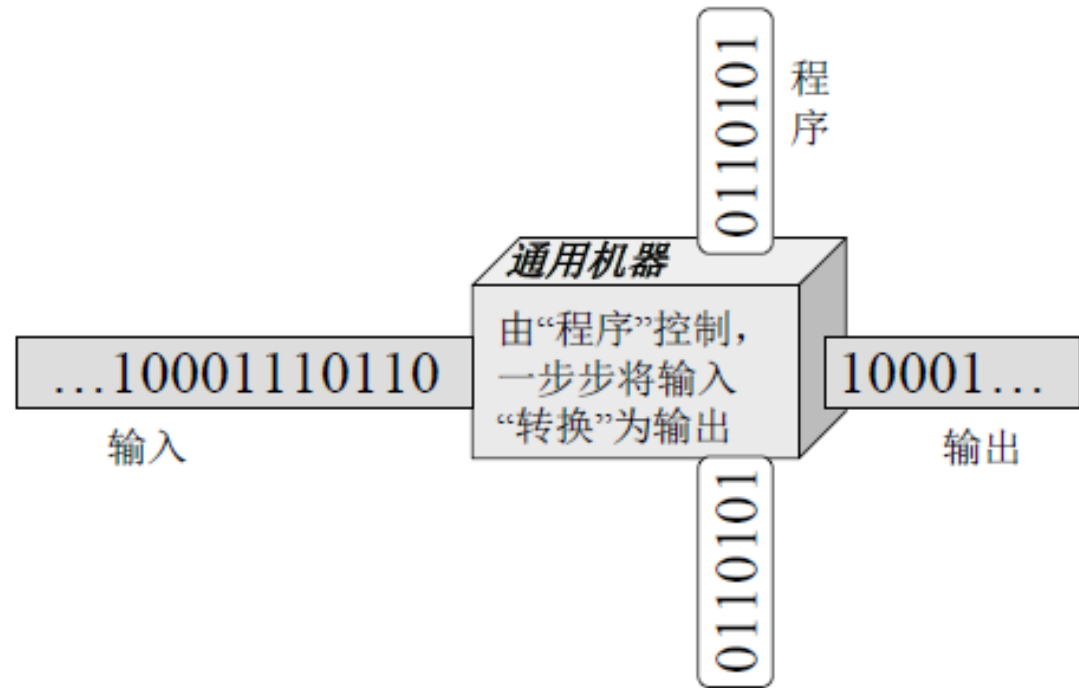
[刷新](#)

Introduction

- What is a computer?
- What can computers do?
- What are the **fundamental capabilities** and **limitations** of computers?

How should we model a computer

Handwritten calculation showing long division of 531 by 8, with intermediate steps and a final result of 66.375.

$$\begin{array}{r} 66.375 \\ 8 \overline{) 531} \\ \underline{48} \\ 51 \\ \underline{48} \\ 30 \\ \underline{24} \\ 6 \end{array}$$


参考文献

- Michael Sipser, Introduction to the Theory of Computation(3rd Edition), Cengage Learning, 2012
- Harry R. Lewis, Christos H. Papadimitriou, Elements of the Theory of Computation (Second Edition), Prentice Hall(清华大学出版社), 1999

Introduction

- Automata Theory
- Computability Theory
- Complexity Theory

Complexity Theory (1)

- Computer problems come in different varieties; some are **easy**, and some are **hard**
 - *Sorting problem*: arrange a list of numbers in given order
 - *Scheduling problem*: find a schedule of classes for the entire university
- What makes some problems **computationally hard** and others easy?
 - Scheme for classifying problems according to their **computational difficulty**

Complexity Theory (2)

- When **confronted** a problem that appears to be computational **hard**
 - By understanding which aspect of the problem is at the **root** of the difficulty
 - Finding solutions that only **approximate** the perfect one
 - Some problems are hard only in the **worst case** situation, but easy most of the time
 - Considering an **alternative** types of computation

Complexity Theory (3)

- One applied area of complexity theory – **cryptography**
 - In most fields, an **easy** computational problem is preferable to a **hard** one because easy ones are cheaper to solve
 - **Cryptography** is unusual because it specifically requires computational problems that are hard, rather than easy

Computability Theory (1)

- During the **first half** of the twentieth century, mathematicians such as **Kurt Godel**, **Alan Turing**, and **Alonzo Church** discovered that certain basic problems cannot be solved by computers
 - Determining whether a **mathematical statement**(数学命题) is true or false
- Development of **theoretical models** of computers
 - **Foundation** of the construction of actual computers

Computability Theory (2)

- The theories of computability and complexity are **closely related**
 - In complexity theory, the objective is to classify problems as **easy** ones and **hard** ones; whereas in computability theory, the classification of problems is by those that are **solvable** and those that are not
 - Computability theory introduces several of the concepts used in complexity theory

Automata Theory

- Automata theory deals with the **definitions and properties** of **mathematical models of computation**.
 - *Finite automaton* - used in text processing, compilers, and hardware design
 - *Context-free grammar* - used in programming languages and artificial intelligence
- Automata theory is an excellent place to begin the study of the theory of computation. The theories of computability and complexity require a **precise definition** of a *computer*

Mathematical Notations and Terminology

- Sets
- Sequences and tuples
- Functions and relations
- Graphs
- Strings and languages
- Boolean logic

Sets(1)

- A **set** is a **group** of objects represented as a unit
- Sets may contain any type of object, including numbers, symbols, and even other sets
- The objects in a set are called its **elements** or **members**

Sets(2)

- Sets may be described **formally** in several ways
 - By listing a set's elements inside braces
 - By describing a set containing elements according to some rules

$$S = \{7, 21, 57\}$$

$$S = \{n \mid n = m^2 \text{ for some } m \in \mathcal{N}\}$$

Sets(3)

Set membership and nonmembership : $\in$, $\notin$

Subset and **proper** subset : $\subseteq$

Finite set and **infinite** set

Empty set : $\emptyset$

Singleton set: a set with one member

$$\mathbb{N} = \{1, 2, 3, \dots\}$$

$$\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$$

Set Operations(4)

Union: $A \cup B$

Intersection: $A \cap B$

Complement: $\overline{A}$

Power set of A is the set of all subset of A

Sequences and Tuples(1)

A *Sequence* of objects is a list of these objects in some order

$(7, 21, 57)$ and $(57, 21, 7)$

Sequences may be finite or infinite

Finite sequences often are called *tuples*(元组)

A sequence with k elements is a *k-tuple*(k 元组)

A 2-tuple is also called an *ordered pair*(有序对)

Sequences and Tuples(2)

- $A \times B$ is the set of all ordered pairs wherein the first element is a member of A and the second element is a member of B
- Cartesian product or Cross product

$$A = \{1, 2\} \text{ and } B = \{x, y, z\}$$

$$A \times B = \{(1, x), (1, y), (1, z), (2, x), (2, y), (2, z)\}$$

Sequences and Tuples(2)

- Cartesian product of k sets, $A_1, A_2, \dots, A_k$, written $A_1 \times A_2 \times \dots \times A_k$.
- It is the set consisting of all k -tuples $(a_1, a_2, \dots, a_k)$ where $a_i \in A_i$

$$A = \{1, 2\} \text{ and } B = \{x, y, z\}$$

$$A \times B \times A = \{(1, x, 1), (1, x, 2), (1, y, 1), (1, y, 2), (1, z, 1), (1, z, 2), (2, x, 1), (2, x, 2), (2, y, 1), (2, y, 2), (2, z, 1), (2, z, 2)\}$$

$$\overbrace{A \times A \times \dots \times A}^k = A^k$$

Functions and Relations(1)

- A function is an object that set up an **input-output relationship**
- A function takes an input and produces an output
- If f is a function whose output value is b when the input value is a , we write $f(a)=b$
- The set of possible inputs to the function is called its **domain(定义域)**
- The outputs of a function come from a set called its **range(值域)**

Functions and Relations(2)

- The notation for saying that *f* is function with domain *D* and range *R* is

$$f : D \rightarrow R$$

$$abs : \mathbb{Z} \rightarrow \mathbb{Z}$$

$$add : \mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z}$$

Functions and Relations(3)

- Consider the function $f: \{0, 1, 2, 3, 4\} \rightarrow \{0, 1, 2, 3, 4\}$

n	$f(n)$
0	1
1	2
2	3
3	4
4	0

- This function adds 1 to its input and then output the result modulo 5. Let

$$Z_m = \{0, 1, 2, \dots, m-1\}$$

- The aforementioned function f has the form

$$f: Z_5 \rightarrow Z_5$$

Functions and Relations(4)

$$g : Z_4 \times Z_4 \rightarrow Z_4$$

g	0	1	2	3
0	0	1	2	3
1	1	2	3	0
2	2	3	0	1
3	3	0	1	2

- The function g is the addition function modulo 4

Functions and Relations(5)

- When the domain of a function f is $A_1 \times A_2 \times \dots \times A_k$ for some sets $A_1, A_2, \dots, A_k$, the input to f is a k -tuple $(a_1, a_2, \dots, a_k)$ and we call the a_i the arguments to f (a_i 是 f 的自变量)
- A function with k arguments is called a k -ary function (k 元函数) and k is called the **arity** of the function (k 称作函数的元数)
- Infix notation($中缀表示法$): $a + b$
- Prefix notation($前缀表示法$): $\text{add}(a, b)$

Functions and Relations(6)

- A **predicate**(谓词) or **property**(性质) is a function whose range is $\{\text{TRUE}, \text{FALSE}\}$
- For example, let *even* be a property that is TRUE if its input is an even number and FALSE if its input is an odd number. Thus $\text{even}(4) = \text{TRUE}$ and $\text{even}(5) = \text{FALSE}$
- A property whose domain is a set of k -tuple $A \times A \times \dots \times A$ is called a *relation*, a *k-ary relation*
- *2 - ary relation, binary relation*

Functions and Relations(7)

- When writing an expression involving a **binary relation**, we customarily use **infix notation**
 - For example, “less than” is a relation usually written with the **infix operation symbol** $<$
 - “Equality”, written with the $=$ symbol, is another familiar relation
- If R is a binary relation, the statement aRb means that $aRb = \text{TRUE}$. Similarly, if R is a k -ary relation, the statement $R(a_1, a_2, \dots, a_k)$ means that $R(a_1, a_2, \dots, a_k) = \text{TRUE}$

Functions and Relations(8)

- A special type of binary relation, called an **equivalence relation**, captures the notion of two objects being equal in some feature.
- A binary relation R is an equivalence relation if R satisfies **three conditions**
 - 1. R is **reflexive(自反)** if for every x , xRx ;
 - 2. R is **symmetric(对称)** if for every x and y , xRy implies yRx ; and
 - 3. R is **transitive(传递)** if for every x , y , and z , xRy and yRz implies xRz

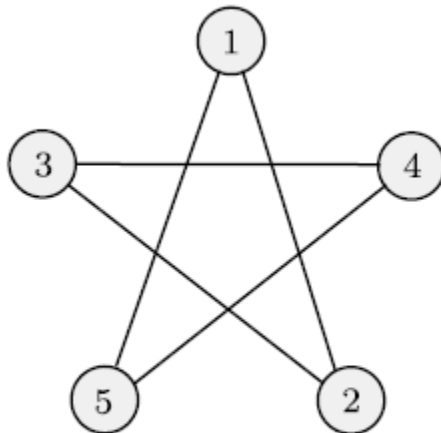
Functions and Relations(9)

EXAMPLE 0.11

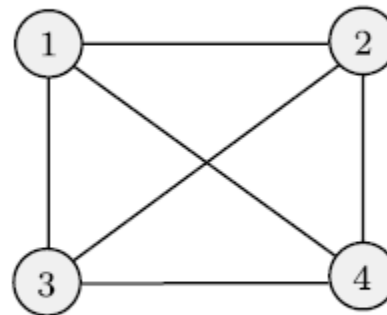
Define an equivalence relation on the natural numbers, written $\equiv_7$. For $i, j \in \mathcal{N}$, say that $i \equiv_7 j$, if $i - j$ is a multiple of 7. This is an equivalence relation because it satisfies the three conditions. First, it is reflexive, as $i - i = 0$, which is a multiple of 7. Second, it is symmetric, as $i - j$ is a multiple of 7 if $j - i$ is a multiple of 7. Third, it is transitive, as whenever $i - j$ is a multiple of 7 and $j - k$ is a multiple of 7, then $i - k = (i - j) + (j - k)$ is the sum of two multiples of 7 and hence a multiple of 7, too. ■

Graphs(1)

- An **undirected graph**, *or* simply a **graph**, is a set of points with lines connecting some of the points
- The points are called **nodes** *or* **vertices**, and the lines are called **edges**
- The number of edges at a particular node is the **degree** of that node



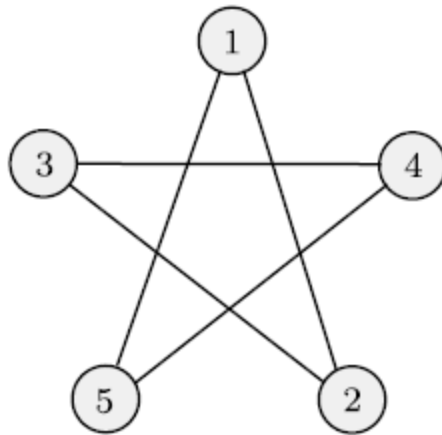
(a)



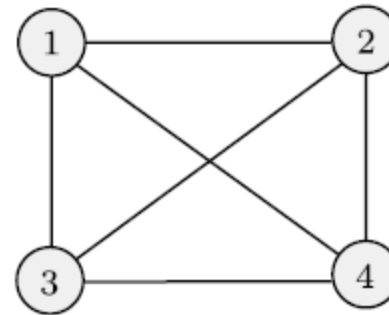
(b)

Graphs(2)

- In a graph **G** that contains nodes i and j , the pair (i, j) represents the **edge** that connects i and j
- If V is the set of nodes of G and E is the set of edges, we say **$G = (V, E)$**



(a)



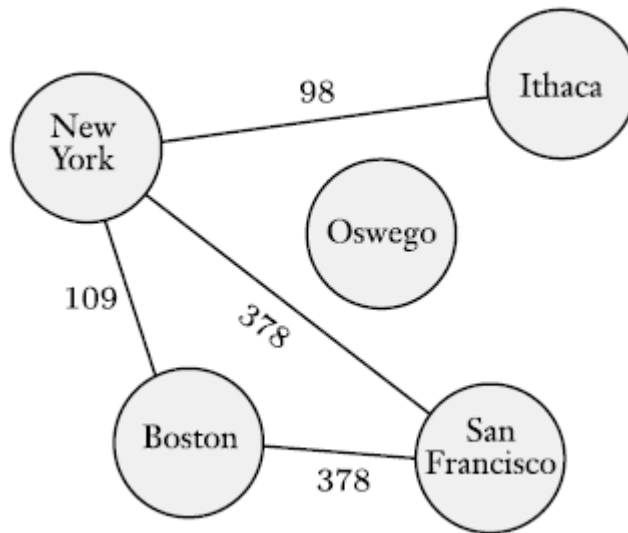
(b)

$(\{1, 2, 3, 4, 5\}, \{(1, 2), (2, 3), (3, 4), (4, 5), (5, 1)\})$

$(\{1, 2, 3, 4\}, \{(1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4)\})$

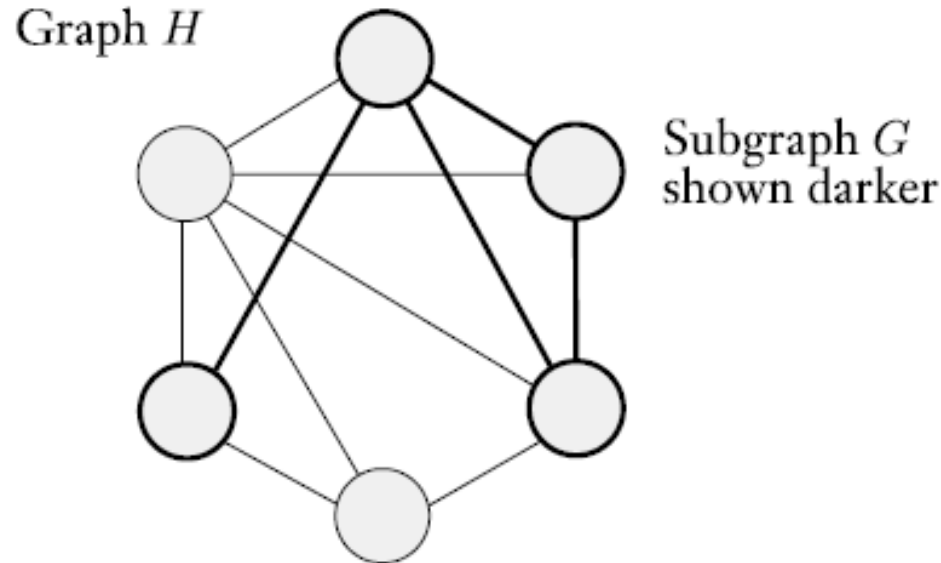
Graphs(3)

- labeled graph(标号图、标定图)



Graphs(4)

- Graph G is a **subgraph** of graph H if the nodes of G are a subset of the nodes of H , and the edges of G are the edges of H on the corresponding nodes

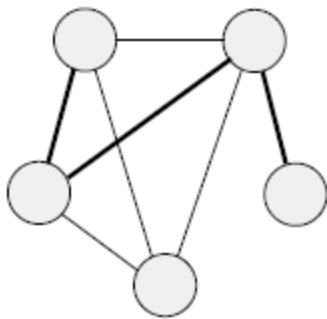


Graphs(5)

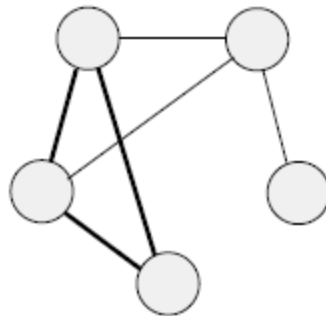
- A **path** in a graph is a sequence of nodes connected by edges
- A **simple path** is a path that doesn't repeat any nodes.
- A graph is **connected** if every two nodes have a path between them
- A path is a **cycle** if it starts and ends in the same node

Graphs(6)

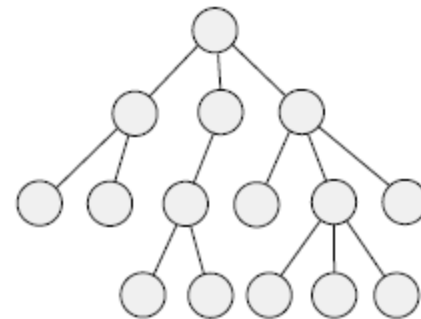
- A **simple cycle** is one that contains at least three nodes and repeats only the first and last nodes
- A graph is a **tree** if it is connected and has no simple cycles



(a)



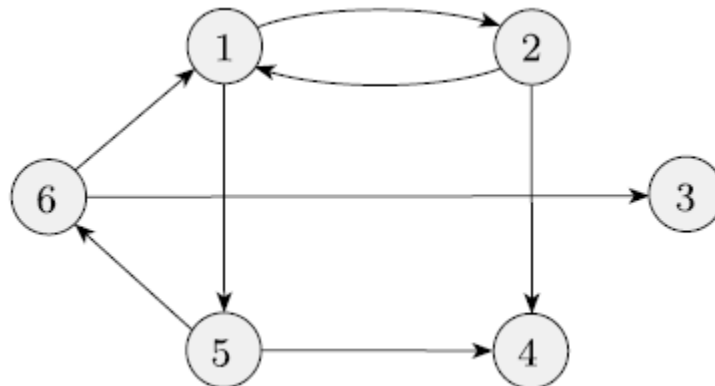
(b)



(c)

Graphs(7)

- A **directed graph**(有向图) has arrows instead of lines
- The number of arrows pointing from a particular node is the **outdegree**(出度) of that node
- the number of arrows pointing to a particular node is the **indegree**(入度)



Strings and Languages(1)

- **Strings of characters** are fundamental building blocks in computer science
- The **alphabet(字母表)** over which the strings are defined may vary with the application
- **Alphabet** - any nonempty finite set
- The members of the alphabet are the **symbols(符号)** of the alphabet
- Capital Greek letters Σ and Γ to designate **alphabets** and a typewriter font for **symbols** from an alphabet

Strings and Languages(2)

- $\Sigma_1 = \{0, 1\}$
- $\Sigma_2 = \{a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z\}$
- $\Gamma = \{0, 1, x, y, z\}$
- A **string over an alphabet** is a **finite sequence of symbols** from that alphabet, usually written next to one another and not separated by commas
- **01001** is a string over Σ_1
- **abracadabra** is a string over Σ_2

Strings and Languages(3)

- If ω is a string over Σ , the **length** of ω , written $|\omega|$, is the number of symbols that it contains
- The string of length **zero** is called the **empty string** and is written ε
- If ω has length n , we can write $\omega = \omega_1\omega_2 \cdots \omega_n$ where each $\omega_i \in \Sigma$. The **reverse** of ω , written ω^R , is the string obtained by writing ω in the opposite order (i.e., $\omega_n\omega_{n-1} \cdots \omega_1$)
- String z is a **substring** of ω if z appears consecutively within ω . For example, **cad** is a substring of **abracadabra**

Strings and Languages(4)

- If we have string x of length m and string y of length n , the **concatenation** of x and y , written xy , is the string obtained by appending y to the end of x , as in $x_1 \cdots x_m y_1 \cdots y_n$
- To concatenate a string with itself many times, we use the superscript notation x^k to mean

$$\overbrace{xx \cdots x}^k$$

Strings and Languages(5)

- The **lexicographic order**(字典序) of strings is the same as the familiar dictionary order
- We'll occasionally use a modified lexicographic order, called **shortlex order** or simply **string order**, that is identical to lexicographic order, except that shorter strings **precede** longer strings
- Thus the string ordering of all strings over the alphabet $\{0,1\}$ is $(\epsilon, 0, 1, 00, 01, 10, 11, 000, \dots)$

Strings and Languages(6)

- String x is a **prefix** of string y if a string z exists where $xz = y$, and that x is a **proper prefix** of y if in addition $x \neq y$
- A **language** is a set of strings
- A language is **prefix-free** if no member is a proper prefix of another member

Boolean Logic(1)

- **Boolean logic** is a mathematical system built around the two values TRUE and FALSE
- The values TRUE and FALSE are called the **Boolean values** and are often represented by the values 1 and 0
- We can manipulate Boolean values with the **Boolean operations**
- Conjunction(合取), AND, $\wedge$
- Disjunction(析取), OR, $\vee$
- Negation(非), NOT, $\neg$

Boolean Logic(2)

$$0 \wedge 0 = 0$$

$$0 \vee 0 = 0$$

$$\neg 0 = 1$$

$$0 \wedge 1 = 0$$

$$0 \vee 1 = 1$$

$$\neg 1 = 0$$

$$1 \wedge 0 = 0$$

$$1 \vee 0 = 1$$

$$1 \wedge 1 = 1$$

$$1 \vee 1 = 1$$

Boolean Logic(3)

- We use **Boolean operations** for combining simple statements into more complex **Boolean expressions**, just as we use the arithmetic operations $+$ and $\times$ to construct complex **arithmetic expressions**
- For example, if P is the Boolean value representing the truth of the statement "the sun is shining" and Q represents the truth of the statement "today is Monday", we may write $P \wedge Q$ to represent the truth value of the statement "the sun is shining *and today is Monday*"

Boolean Logic(4)

- Exclusive OR(异或), XOR, $\oplus$
- Equality(等价), $\leftrightarrow$
- Implication(蕴含), $\rightarrow$

$$0 \oplus 0 = 0$$

$$0 \oplus 1 = 1$$

$$1 \oplus 0 = 1$$

$$1 \oplus 1 = 0$$

$$0 \leftrightarrow 0 = 1$$

$$0 \leftrightarrow 1 = 0$$

$$1 \leftrightarrow 0 = 0$$

$$1 \leftrightarrow 1 = 1$$

$$0 \rightarrow 0 = 1$$

$$0 \rightarrow 1 = 1$$

$$1 \rightarrow 0 = 0$$

$$1 \rightarrow 1 = 1$$

Boolean Logic(5)

$$\begin{array}{ll} P \vee Q & \neg(\neg P \wedge \neg Q) \\ P \rightarrow Q & \neg P \vee Q \\ P \leftrightarrow Q & (P \rightarrow Q) \wedge (Q \rightarrow P) \\ P \oplus Q & \neg(P \leftrightarrow Q) \end{array}$$

$$P \wedge (Q \vee R) \text{ equals } (P \wedge Q) \vee (P \wedge R)$$

$$P \vee (Q \wedge R) \text{ equals } (P \vee Q) \wedge (P \vee R)$$

distributive law

Definitions, Theorems and Proofs(1)

- **Definitions** describe the objects and notions that we use
- A **mathematical statements** expresses that some object has a certain property
- A **proof** is a convincing logical argument that a statement is true
- A **theorem** is a mathematical statement proved true

Definitions, Theorems and Proofs(2)

- Occasionally we prove statements that are interesting only because they assist in the proof of another, more significant statement. Such statements are called **lemmas**(引理)
- Occasionally a theorem or its proof may allow us to conclude easily that other, related statements are true. These statements are called **corollaries**(推论) of the theorem

Definitions, Theorems and Proofs(3)

- Finding proofs
 - P if and only if Q ($P \text{ iff } Q$)
- Forward direction
 - if P is true, then Q is true
- Reverse direction
 - If Q is true, then P is true
- Counterexample

Types of Proof(1)

- **Proof by construction(构造性证明)**
 - Many theorems state that a particular type of object exists. One way to prove such a theorem is by demonstrating how to construct the object
- **Proof by contradiction(反证)**
 - In one common form of argument for proving a theorem, we assume that the theorem is false and then show that this assumption leads to an obviously false consequence

Types of Proof(2)

- Proof by induction(归纳法)
 - Proof by induction is an advanced method used to show that all elements of an infinite set have a specified property
- Basis(归纳基础)
 - The basis proves that $\mathcal{P}(1)$ is true
- Induction steps(归纳步骤)
 - The induction step proves that for each $i \geq 1$, if $\mathcal{P}(i)$ is true, then so is $\mathcal{P}(i+1)$
- Induction hypothesis(归纳假设)